

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [2]: # Section B - Task 1: Delivery Fee Array Calculator

np.random.seed(42)

distances = np.random.uniform(1.0, 15.0, 25)
fees = 20 + 5 * distances

mask = fees > 60
qualifying_orders = np.column_stack((distances[mask], fees[mask]))

print("Orders with delivery fee above Rs 60:")
print("Distance (km)    Fee (Rs)")
print(np.round(qualifying_orders, 2))

print("\nFee Statistics")
print(f"Minimum: {np.min(fees):.2f}")
print(f"Maximum: {np.max(fees):.2f}")
print(f"Mean:      {np.mean(fees):.2f}")
print(f"Std Dev:   {np.std(fees):.2f}")
```

Orders with delivery fee above Rs 60:
Distance (km) Fee (Rs)
[[14.31 91.55]
 [11.25 76.24]
 [9.38 66.91]
 [13.13 85.63]
 [9.42 67.08]
 [10.91 74.57]
 [14.58 92.89]
 [12.65 83.27]
 [8.35 61.73]
 [9.57 67.83]]

Fee Statistics
Minimum: 26.44
Maximum: 92.89
Mean: 55.86
Std Dev: 19.56

```
In [3]: # Section B - Task 2: Restaurant Performance Analyser

restaurants = (["Spice Hub"] * 10 + ["Urban Bites"] * 10 + ["Tasty Corner"] * 10 + ["Food Street"] * 10 + ["Royal Kitchen"] * 10)
cities = (["Vadodara", "Ahmedabad", "Surat"] * 16 + ["Vadodara", "Ahmedabad"])
cuisines = (["Indian", "Chinese", "Fast Food"] * 16 + ["Indian", "Chinese"])

order_values = ([650, 700, 720, 680, 750, 710, 690, 730, 760, 700] + [800, 780, 820, 790, 810, 830, 770, 805, 795, 815] +
[500, 520, 480, 510, 530, 490, 515, 505, 525, 495] + [900, 880, 920, 910, 890, 930, 870, 905, 895, 915] +
[600, 620, 580, 610, 630, 590, 615, 605, 625, 595])

delivery_times = ([30, 31, 29, 32, 30, 28, 31, 29, 30, 32] + [34, 33, 35, 34, 32, 33, 31, 34, 33, 32] +
[38, 40, 37, 39, 41, 38, 40, 37, 39, 38] + [29, 30, 28, 31, 30, 29, 32, 30, 28, 31] + [36, 37, 35, 38, 36, 37, 34, 36, 35, 37])

ratings = ([4.4, 4.5, 4.3, 4.6, 4.4, 4.5, 4.2, 4.6, 4.5, 4.4] + [4.1, 4.2, 4.0, 4.3, 4.1, 4.2, 4.0, 4.1, 4.2, 4.1] +
[3.8, 3.9, 4.0, 3.7, 3.9, 3.8, 3.9, 4.0, 3.8, 3.9] + [4.6, 4.7, 4.5, 4.8, 4.6, 4.7, 4.5, 4.8, 4.7, 4.6] +
[3.9, 4.0, 3.8, 4.1, 3.9, 4.0, 3.7, 3.9, 4.0, 3.8])

df = pd.DataFrame({"restaurant_name": restaurants, "city": cities, "order_value": order_values, "delivery_time_mins": delivery_times,
"rating": ratings, "cuisine_type": cuisines})
```

```
In [4]: df
```

Out[4]:

	restaurant_name	city	order_value	delivery_time_mins	rating	cuisine_type
0	Spice Hub	Vadodara	650	30	4.4	Indian
1	Spice Hub	Ahmedabad	700	31	4.5	Chinese
2	Spice Hub	Surat	720	29	4.3	Fast Food
3	Spice Hub	Vadodara	680	32	4.6	Indian
4	Spice Hub	Ahmedabad	750	30	4.4	Chinese
5	Spice Hub	Surat	710	28	4.5	Fast Food
6	Spice Hub	Vadodara	690	31	4.2	Indian
7	Spice Hub	Ahmedabad	730	29	4.6	Chinese
8	Spice Hub	Surat	760	30	4.5	Fast Food
9	Spice Hub	Vadodara	700	32	4.4	Indian
10	Urban Bites	Ahmedabad	800	34	4.1	Chinese
11	Urban Bites	Surat	780	33	4.2	Fast Food
12	Urban Bites	Vadodara	820	35	4.0	Indian
13	Urban Bites	Ahmedabad	790	34	4.3	Chinese
14	Urban Bites	Surat	810	32	4.1	Fast Food
15	Urban Bites	Vadodara	830	33	4.2	Indian
16	Urban Bites	Ahmedabad	770	31	4.0	Chinese
17	Urban Bites	Surat	805	34	4.1	Fast Food
18	Urban Bites	Vadodara	795	33	4.2	Indian
19	Urban Bites	Ahmedabad	815	32	4.1	Chinese
20	Tasty Corner	Surat	500	38	3.8	Fast Food
21	Tasty Corner	Vadodara	520	40	3.9	Indian
22	Tasty Corner	Ahmedabad	480	37	4.0	Chinese
23	Tasty Corner	Surat	510	39	3.7	Fast Food
24	Tasty Corner	Vadodara	530	41	3.9	Indian
25	Tasty Corner	Ahmedabad	490	38	3.8	Chinese
26	Tasty Corner	Surat	515	40	3.9	Fast Food
27	Tasty Corner	Vadodara	505	37	4.0	Indian
28	Tasty Corner	Ahmedabad	525	39	3.8	Chinese
29	Tasty Corner	Surat	495	38	3.9	Fast Food
30	Food Street	Vadodara	900	29	4.6	Indian
31	Food Street	Ahmedabad	880	30	4.7	Chinese
32	Food Street	Surat	920	28	4.5	Fast Food
33	Food Street	Vadodara	910	31	4.8	Indian
34	Food Street	Ahmedabad	890	30	4.6	Chinese
35	Food Street	Surat	930	29	4.7	Fast Food
36	Food Street	Vadodara	870	32	4.5	Indian
37	Food Street	Ahmedabad	905	30	4.8	Chinese
38	Food Street	Surat	895	28	4.7	Fast Food
39	Food Street	Vadodara	915	31	4.6	Indian
40	Royal Kitchen	Ahmedabad	600	36	3.9	Chinese
41	Royal Kitchen	Surat	620	37	4.0	Fast Food
42	Royal Kitchen	Vadodara	580	35	3.8	Indian
43	Royal Kitchen	Ahmedabad	610	38	4.1	Chinese
44	Royal Kitchen	Surat	630	36	3.9	Fast Food
45	Royal Kitchen	Vadodara	590	37	4.0	Indian
46	Royal Kitchen	Ahmedabad	615	34	3.7	Chinese
47	Royal Kitchen	Surat	605	36	3.9	Fast Food
48	Royal Kitchen	Vadodara	625	35	4.0	Indian
49	Royal Kitchen	Ahmedabad	595	37	3.8	Chinese

In [6]:

```
result = (df.groupby("restaurant_name").agg(mean_order_value=("order_value", "mean"),mean_delivery_time=("delivery_time_mins", "mean"),
mean_rating=("rating", "mean")))

result = result[(result["mean_rating"] > 4.0) &(result["mean_delivery_time"] < 35)]

result = (result.sort_values("mean_order_value", ascending=False).reset_index())

print(result.to_string(index=False, float_format=lambda x: f"{x:.2f}"))
```

restaurant_name	mean_order_value	mean_delivery_time	mean_rating
Food Street	901.50	29.80	4.65
Urban Bites	801.50	33.10	4.13
Spice Hub	709.00	30.20	4.44

In [12]:

```
# Section B - Task 3: Monthly Order Trend Dashboard

np.random.seed(20)

months = np.arange(1, 13)
orders = np.random.randint(1000, 5001, 12)
average_order_value = np.random.uniform(200, 400, 12)
revenue = orders * average_order_value

delivery_mean = 28
delivery_std = 4
delivery_times = np.random.normal(delivery_mean, delivery_std, 500)

fig, axes = plt.subplots(1, 3, figsize=(16, 5))

# SubPlot 1: Orders

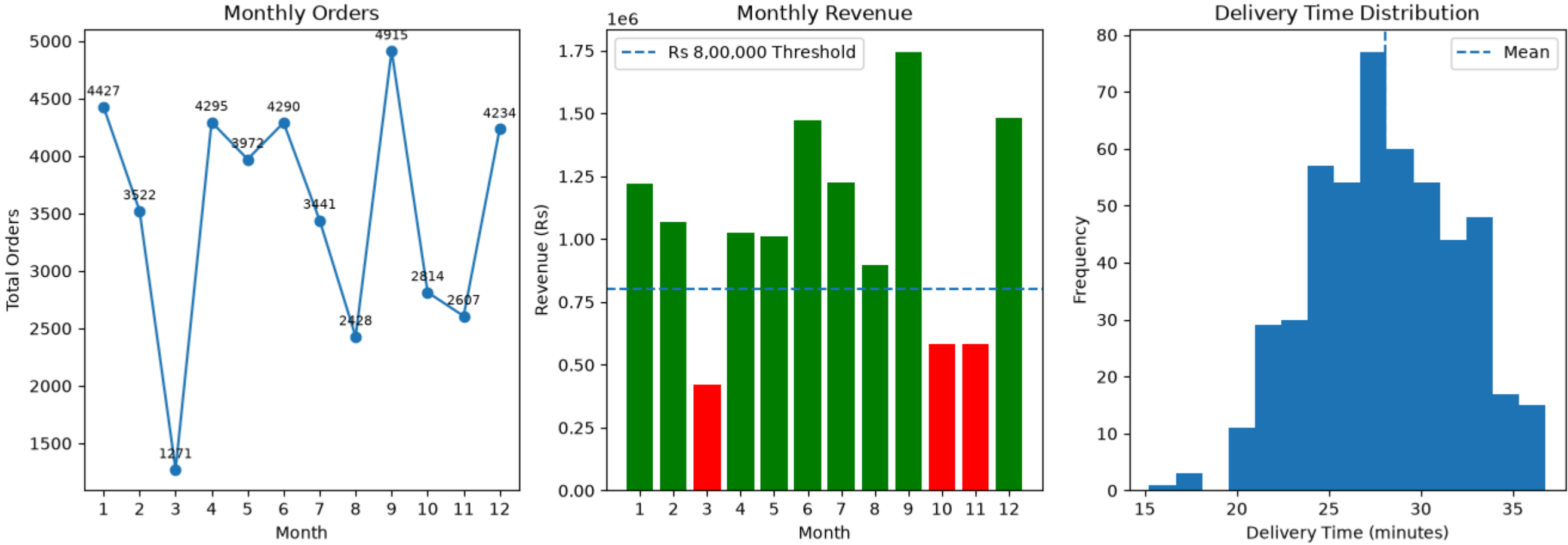
axes[0].plot(months, orders, marker="o")
for month, order in zip(months, orders):
```

```
axes[0].text(month, order + 100, str(order), ha="center", fontsize=8)
axes[0].set_title("Monthly Orders")
axes[0].set_xlabel("Month")
axes[0].set_ylabel("Total Orders")
axes[0].set_xticks(months)

# Subplot 2: Revenue
bar_colors = ["green" if value > 800000 else "red" for value in revenue]
axes[1].bar(months, revenue, color=bar_colors)
axes[1].axhline(800000, linestyle="--", label="Rs 8,00,000 Threshold")
axes[1].set_title("Monthly Revenue")
axes[1].set_xlabel("Month")
axes[1].set_ylabel("Revenue (Rs)")
axes[1].set_xticks(months)
axes[1].legend()

# Subplot 3: Delivery time distribution
axes[2].hist(delivery_times, bins=15)
axes[2].axvline(np.mean(delivery_times), linestyle="--", label="Mean")
axes[2].set_title("Delivery Time Distribution")
axes[2].set_xlabel("Delivery Time (minutes)")
axes[2].set_ylabel("Frequency")
axes[2].legend()
```

Out[12]: <matplotlib.legend.Legend at 0x2401fd08cd0>



```
In [17]: fig.suptitle("Food Delivery Monthly Performance Dashboard")
plt.tight_layout()
plt.savefig("food_delivery_dashboard.png", dpi=150, bbox_inches="tight")
plt.close()
print("Dashboard saved as food_delivery_dashboard.png")
```

Dashboard saved as food_delivery_dashboard.png

```
In [18]: # Section B - Task 4: Full EDA Pipeline - Food Delivery Insights

np.random.seed(7)

df = pd.DataFrame({"order_value": np.random.uniform(100, 800, 200), "distance_km": np.random.uniform(1, 20, 200),
"delivery_time_mins": np.random.normal(30, 8, 200), "rating": np.random.uniform(1.0, 5.0, 200),
"discount_pct": np.random.uniform(0, 30, 200)})
```

In [19]: df

Out[19]:

	order_value	distance_km	delivery_time_mins	rating	discount_pct
0	153.415803	15.287118	29.436359	1.610398	17.079069
1	645.943155	2.171641	41.143419	1.247733	4.256927
2	406.886462	15.151650	20.002291	3.285994	18.347489
3	606.425624	18.979243	18.019293	1.749068	29.233717
4	784.592658	12.467631	24.767687	3.171931	8.202608
...	...	...	...	...	...
195	451.264824	18.616958	14.989387	2.738598	18.982758
196	745.822207	11.297294	33.823406	3.576348	11.126439
197	367.757797	5.591340	27.490924	4.840376	2.385037
198	555.089830	9.306737	23.154610	2.077615	8.599132
199	516.934779	18.006834	33.985596	2.571456	20.803693

200 rows × 5 columns

```
In [21]: # Add 5% null values to delivery time and rating
for column in ["delivery_time_mins", "rating"]:
    missing_rows = np.random.choice(df.index, size=int(0.05 * len(df)), replace=False)
    df.loc[missing_rows, column] = np.nan
```

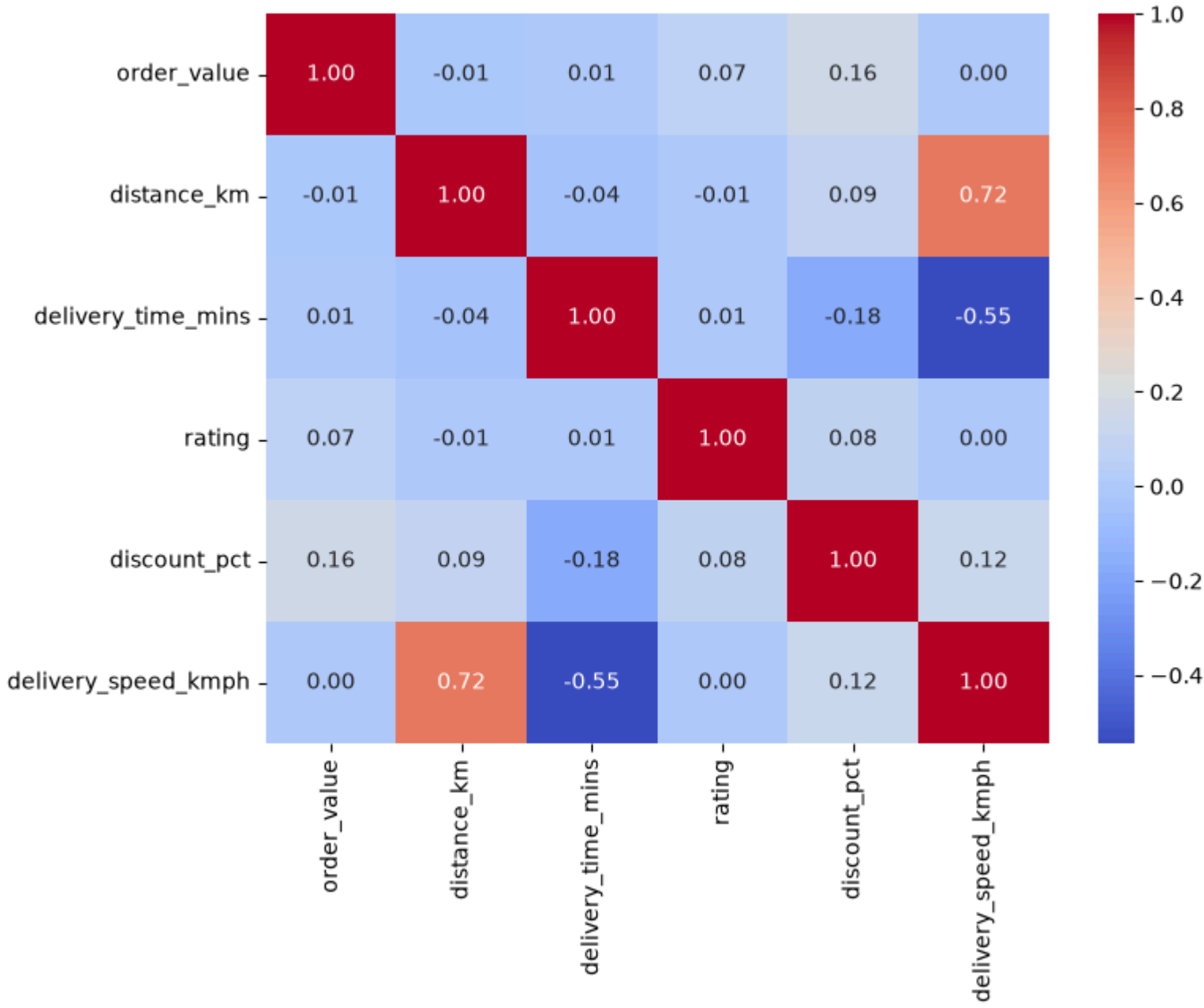
```
In [22]: # Fill missing values with the column median
df["delivery_time_mins"] = df["delivery_time_mins"].fillna(df["delivery_time_mins"].median())
df["rating"] = df["rating"].fillna(df["rating"].median())
```

```
In [23]: # Derived feature
df["delivery_speed_kmph"] = df["distance_km"] / (df["delivery_time_mins"] / 60)
```

```
In [25]: # Three equal-frequency speed bands
df["speed_band"] = pd.qcut(df["delivery_speed_kmph"],q=3,labels=["Slow", "Normal", "Fast"])
```

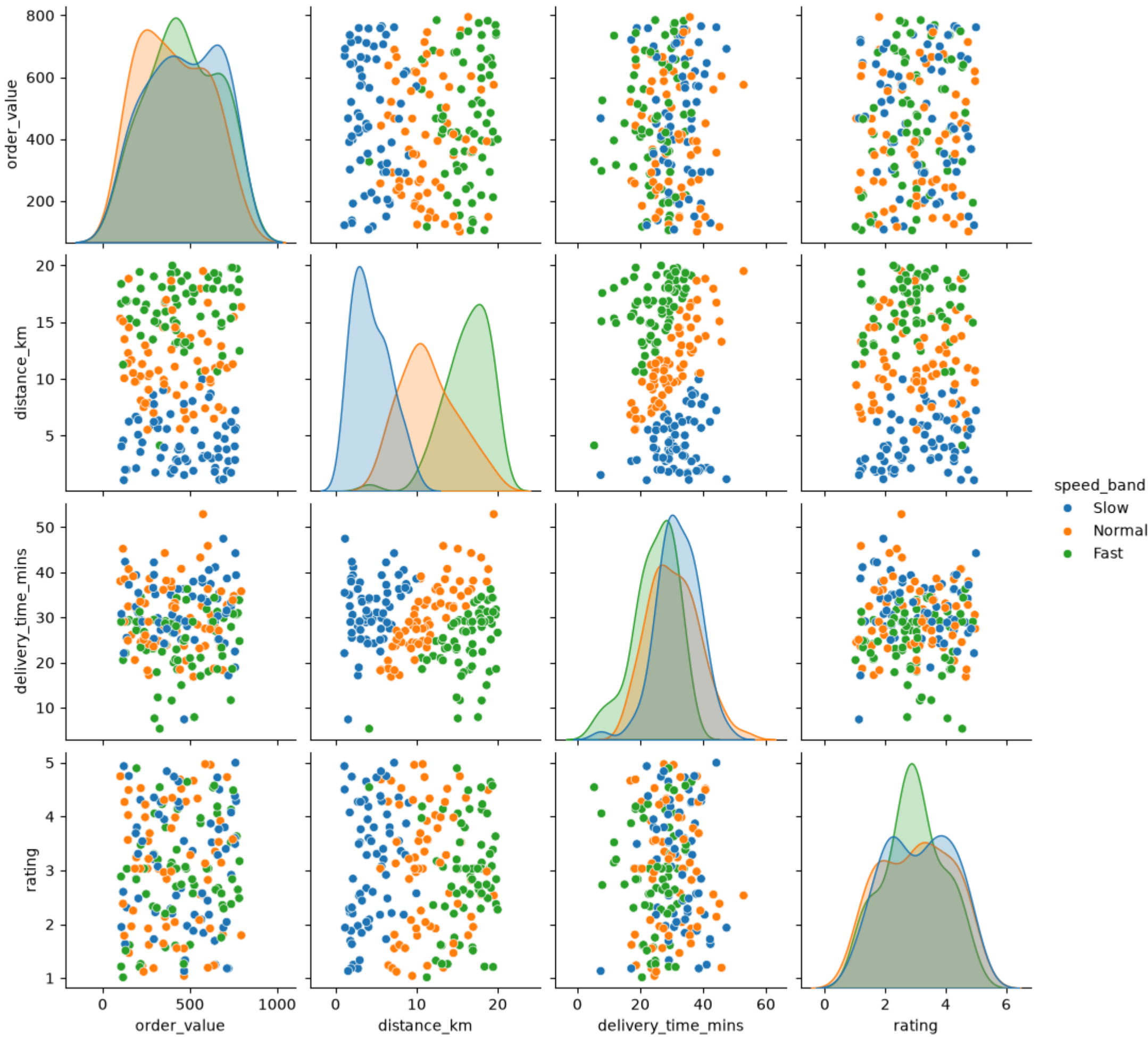
```
In [27]: # Correlation heatmap
numeric_df = df.select_dtypes(include=np.number)
plt.figure(figsize=(8, 6))
sns.heatmap(numeric_df.corr(), annot=True, cmap="coolwarm", fmt=".2f")
```

Out[27]: <Axes: >



```
In [28]: plt.title("Food Delivery Correlation Heatmap")
plt.tight_layout()
plt.savefig("correlation_heatmap.png", dpi=150, bbox_inches="tight")
plt.close()
```

```
In [29]: # Pairplot
pair = sns.pairplot(df[["order_value", "distance_km", "delivery_time_mins", "rating", "speed_band"]], hue="speed_band")
```



```
In [30]: pair.fig.suptitle("Food Delivery Pairplot by Speed Band", y=1.02)
pair.fig.savefig("pairplot.png", dpi=150, bbox_inches="tight")
plt.close("all")
```

```
In [31]: print("EDA completed.")
print("Saved: correlation_heatmap.png")
print("Saved: pairplot.png")
```

EDA completed.
Saved: correlation_heatmap.png
Saved: pairplot.png

In []: